G3-4: SafeRoute

Danial, Akil, Ruy Han, Jeriel, Shri

Why is the classic TSP insufficient?

01

Considers distance only

The classic TSP problem only considers minimising the distance travelled between locations. This does not account for many other possible issues that the driver may encounter.

02

Does not account for driver safety

Especially at different times, locations may become more dangerous than others. For a delivery driver, it could be dangerous to travel all alone in a neighbourhood with a high crime rate at night .

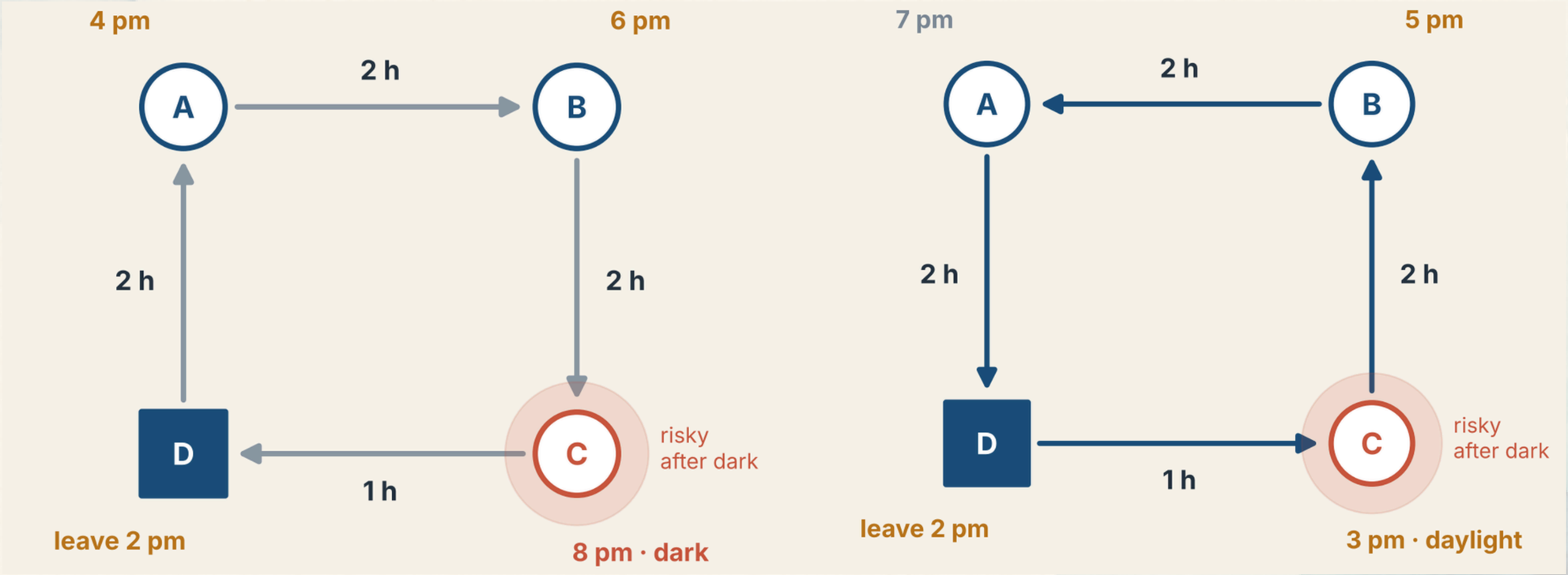
03

Does not account for arrival times

The order in which cities are visited will determines their arrival times. A longer route may be safer if it avoids high-risk locations during the night, and they eventually will have to sleep when it gets too late.

Why is the classic TSP insufficient?

A classic TSP only adds up the edges, so these two routes may look identical. But the **order** of stops decides when you arrive, and arriving at C after dark is riskier.



Data Sources

GeoNames

Fields used

- GeoNameID
- City name
- Latitude/Longitude
- State & Population
- Feature Class
- Feature Code

+

Numbeo

Collected Data

- Day Safety Score
- Night Safety Score
- Crime Index
- Contributor count

Total = 43 cities

geonameid	name	latitude	longitude	feature_class	feature_code	admin1_code	population	crime_index	day_safety	night_safety	day_risk	night_risk	numbeo_contributors	numbeo_last_update	numbeo_url
1736309	Alor Setar	6.12104	100.3601	P	PPLA	02	417800	28.69	80.36	60.71	19.64	39.29	14	22/9/2024	https://www.numbeo.com/crime/in/Alor-Setar
1735168	Ampang	3.15	101.7667	P	PPL	14	126285	39.21	72.5	60	27.5	40	10	15/11/2025	https://www.numbeo.com/crime/in/Ampang-Jaya-Malaysia
1732687	Batu Pahat	1.8548	102.9325	P	PPL	01	156236	24.51	91.67	91.67	8.33	8.33	3	28/9/2021	https://www.numbeo.com/crime/in/Bandar-Penggaram-Malaysia
1734797	Bangi	2.9	101.7833	P	PPL	12	45042	38.57	63.64	52.27	36.36	47.73	11	10/2/2024	https://www.numbeo.com/crime/in/Bangi-Malaysia
1735102	Bayan Lepas	5.29992	100.2605	P	PPL	09	130455	24.45	91.67	61.11	8.33	38.89	9	30/7/2025	https://www.numbeo.com/crime/in/Bayan-Lepas-Malaysia
1737486	Bintulu	3.16667	113.0333	P	PPL	11	151617	38.97	75	58.33	25	41.67	6	16/10/2024	https://www.numbeo.com/crime/in/Bintulu-Malaysia
1735079	Bukit Mertajam	5.36301	100.4667	P	PPL	09	212329	24.59	83.33	65	16.67	35	15	22/7/2025	https://www.numbeo.com/crime/in/Bukit-Mertajam-Malaysia
1735076	Butterworth	5.3991	100.3638	P	PPL	09	107591	24.41	85	65	15	35	5	29/10/2023	https://www.numbeo.com/crime/in/Butterworth-Malaysia
6930887	Cyberjaya	2.92281	101.6572	P	PPL	12	79200	31.99	82.52	55.38	17.48	44.62	36	24/4/2023	https://www.numbeo.com/crime/in/Cyberjaya
1735106	George Town	5.41123	100.3354	P	PPLA	09	158336	24.99	85.96	69.08	14.04	30.92	41	25/2/2026	https://www.numbeo.com/crime/in/George-Town
1734634	Ipoh	4.5841	101.0829	P	PPLA	07	759952	44.8	68.6	38.2	31.4	61.8	37	4/10/2024	https://www.numbeo.com/crime/in/Ipoh
1732752	Johor Bahru	1.4655	103.7578	P	PPLA	01	858118	48.9	62.66	39.46	37.34	60.54	133	30/6/2026	https://www.numbeo.com/crime/in/Johor-Bahru
10941913	Kajang	2.99424	101.7888	P	PPL	12	236240	39.24	67.29	49.64	32.71	50.36	18	8/6/2023	https://www.numbeo.com/crime/in/Kajang-Selangor-Malaysia
1736278	Kangar	6.4414	100.1986	P	PPLA	08	63869	20.88	85	70	15	30	5	14/6/2024	https://www.numbeo.com/crime/in/Kangar-Perlis-Malaysia
1760605	Kepong	3.21398	101.6402	P	PPL	14	85960	36.7	72.73	59.09	27.27	40.91	11	17/10/2024	https://www.numbeo.com/crime/in/Kepong-Malaysia
1732905	Klang	3.03667	101.4433	P	PPL	12	240016	55.98	54.1	37.67	45.9	62.33	89	4/3/2026	https://www.numbeo.com/crime/in/Klang
1736376	Kota Bharu	6.12361	102.2433	P	PPLA	03	568900	41.32	71.88	52.44	28.12	47.56	18	18/6/2024	https://www.numbeo.com/crime/in/Kota-Bharu
12514556	Kota Damansara	3.15454	101.5802	P	PPL	12	500000	29.41	91.67	58.33	8.33	41.67	3	12/4/2022	https://www.numbeo.com/crime/in/Kota-Damansara-Malaysia
1733432	Kota Kinabalu	5.9749	116.0724	P	PPLA	16	500421	44.04	72.73	46.15	27.27	53.85	63	10/3/2025	https://www.numbeo.com/crime/in/Kota-Kinabalu

GeoNames supplies coordinates for **Haversine distance**
Numbeo adds **day/night safety scores** (translated to danger scores) at each stop

Our TSP variant

inputs

- Selected Malaysian cities with coordinates and day/night safety scores
- Starting depot and departure time
- Objective weights (α, β)

output

- An ordered round trip with the lowest combined cost found
- Arrival times and overnight rest schedule
- Total distance, journey time, arrival risk and overall cost

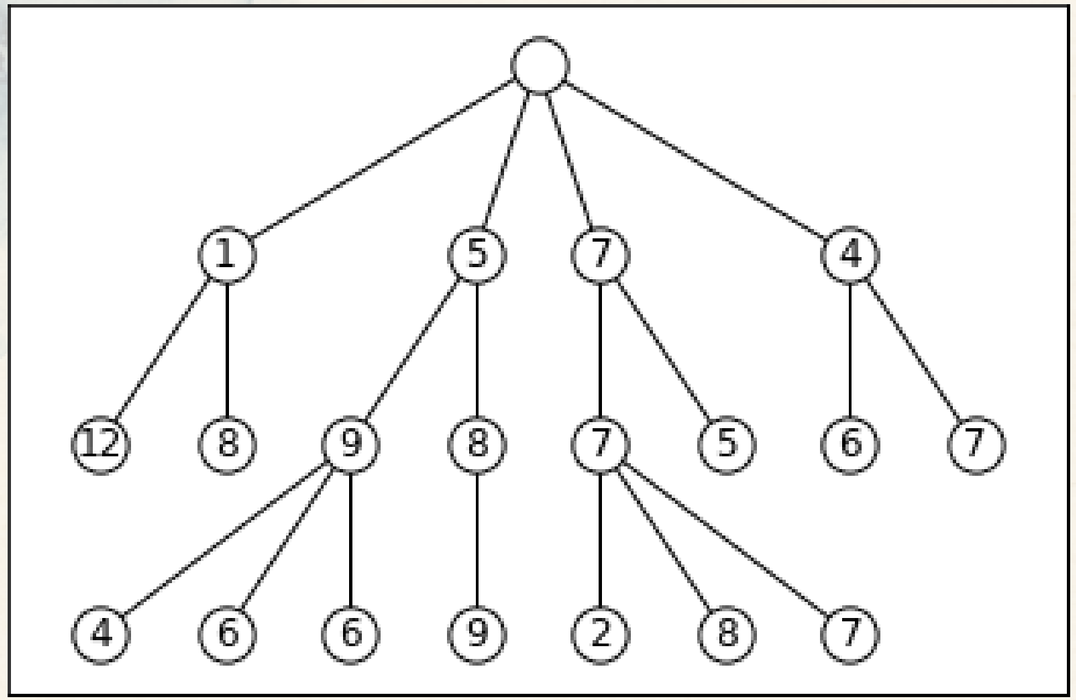
constraints

- Visit each city exactly once and return to the starting depot at the end
- Fixed driving speed of 60km/h and negligible service time
- No nighttime departures - mandatory overnight rests with optional additional rests to reduce arrival risk

$$\text{Cost} = \alpha(\text{Normalized total distance}) + \beta(\text{Normalized total journey time}) + (1 - \alpha - \beta)(\text{Normalized total arrival risk})$$

α and β are weights indicating their assigned importance

Approach A: beam search



idea

Build routes one city at a time, but keep the k cheapest partial routes at every step instead of just one.

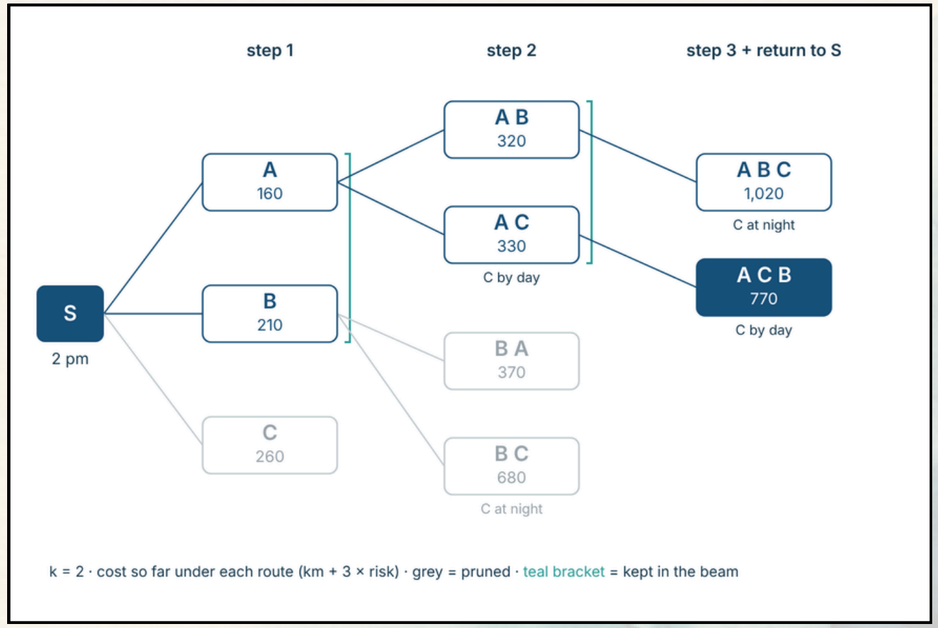
A slightly worse step now can still win later by reaching a risky city before dark.

data structures

- Bounded heap (keeps the top k)
- Parent-pointer tree (routes share their start)
- Visited bitmask (O(1) check)
- Distance matrix

Beam search pseudocode

```
beam = [ start at depot, zero distance / time / risk ]
for depth = 1 to n - 1:
    for each state in beam:
        for each unvisited city c:
            for each departure option (leave now, or rest until 07:00):
                child = extend(state, c)
                if depth = n - 1: add return leg to depot
                push (cost(child), child) into bounded heap of size W
    beam = the W cheapest children
best = beam[0]
route = follow best.parent back to the depot
return evaluate_tour(route)
```

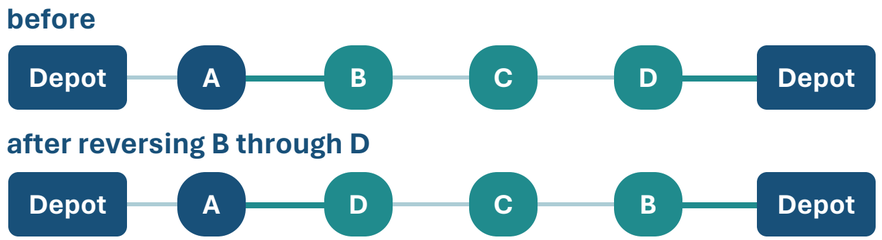


Approach B: 2-opt

idea

Start with a full loop and keep reversals that lower the cost. Stop when no reversal helps.

Default is nearest neighbour by distance with best improvement. ID order is the other start we tested.



data structures

- One tour tuple, with the depot fixed at both ends.
- Each reversal creates a new tuple.
- Update the current tour and cost after a win.
- Shared distance matrix from make_problem().

2-opt pseudocode

```
current = evaluate_tour(start)
repeat:
  best = none, limit = current.cost - 1e-12
  for each pair of stop positions i < j:
    route = reverse(current.tour, i, j)
    trial = evaluate_tour(route)
    if trial.cost < limit:
      best = trial, limit = trial.cost
      if strategy is first: end scan
  if best is none: return current
  current = best
```

A reversal can change later arrivals. We rescore the whole tour and find its cheapest rest schedule.

Complexity

n = selected cities including the depot, W = beam width, k = accepted 2-opt moves

beam search

time

$$O(n^2 + Wn^2 \log(W + 1))$$

Up to W partial routes per step generate $O(Wn^2)$ candidates in total. The heap takes $O(\log(W + 1))$ per candidate.

space

$$O(n^2 + Wn)$$

The distance matrix takes $O(n^2)$. The routes still in the beam share parent pointers and use $O(Wn)$ states.

2-opt

time

$$O(n^2 + (k + 1)n^4)$$

Each scan tries $O(n^2)$ reversals and rescores each in $O(n^2)$. That makes a full scan $O(n^4)$.

space

$$O(n^2)$$

The distance matrix takes $O(n^2)$. The current tour and candidate use $O(n)$ extra space.

Experiment setup

01 same inputs

Same city subsets, starting depot, departure time, objective weights

02 same scoring model

same distance matrix, overnight-rest rules and weighted cost function

03 evaluation metrics

Experiment 1: Mean optimality gap against brute force.

Experiment 2: Relative cost difference across larger datasets.

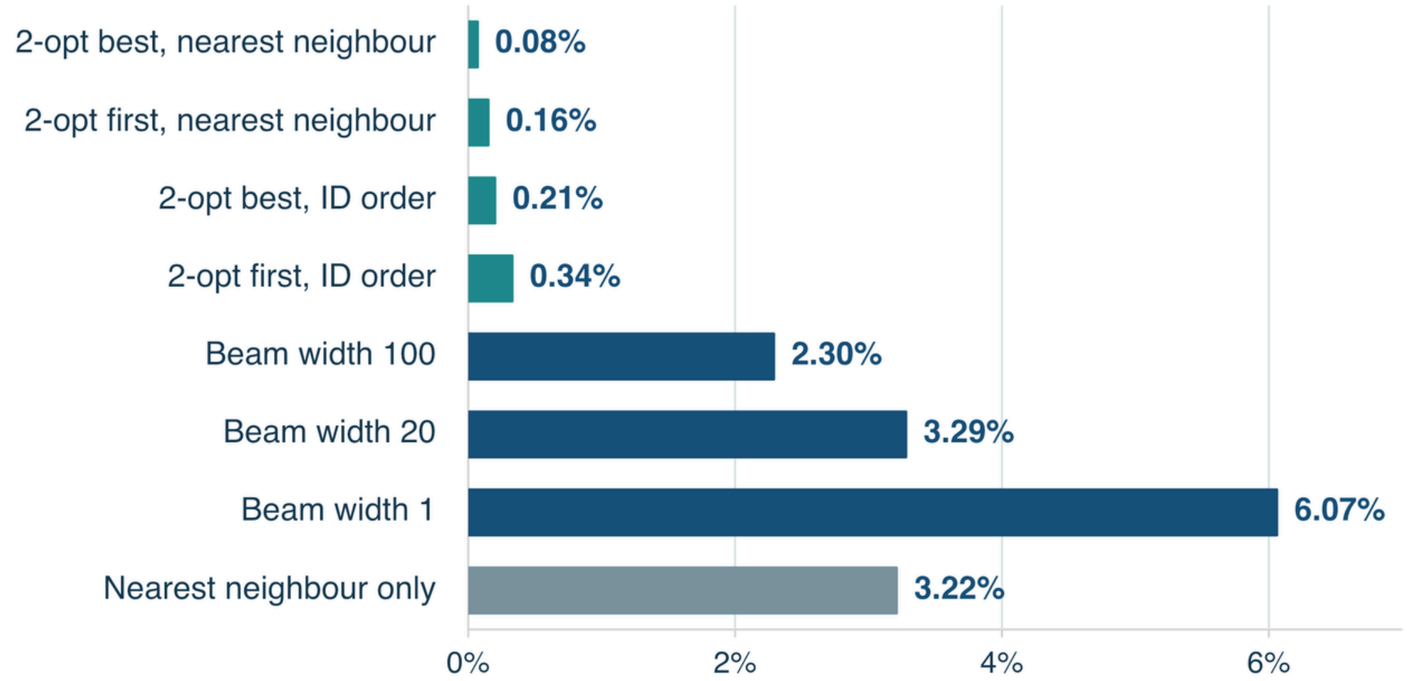
Category	Experiment 1	Experiment 2
Objective	Which algorithm produces solutions closer to the mathematical optimum?	Does 2-opt still outperform Beam Search on larger datasets?
Dataset	200 trials, 9 cities per trial	Klang Valley (6 cities) Peninsular KL (15 cities) Peninsular JB (37 cities) East Malaysia (6 cities) All 43 cities
Weights	Fixed: 40% distance, 20% time, 40% safety	Same fixed weights
Departure Time	06:00, 10:00, 14:00 or 18:00	08:00 and 18:00
Metric	Mean optimality gap (%) **Lower is better**	Percentage by which 2-opt's cost is lower than Beam Search's **Higher means a greater 2-opt advantage**

Experiment 1 results

200 Trials in total were conducted. Default weights (α = 0.4, β = 0.2).

Mean gap to the exact optimum

Lower is better



Main result

Default 2-opt averaged 0.08% above the best cost. Beam width 100 averaged 2.30%.

165 of 200

trials matched the exact optimum

Against beam width 100, 2-opt was cheaper on 141, tied on 55, and costlier on 4.

Experiment 2 results

Default weights (α = 0.4, β = 0.2). Default 2-opt against beam width 100.

Main result

2-opt was cheaper at both times in all four larger cases.

East Malaysia was a tie. Both methods matched the exact optimum at 08:00 and 18:00.

Where did beam win?

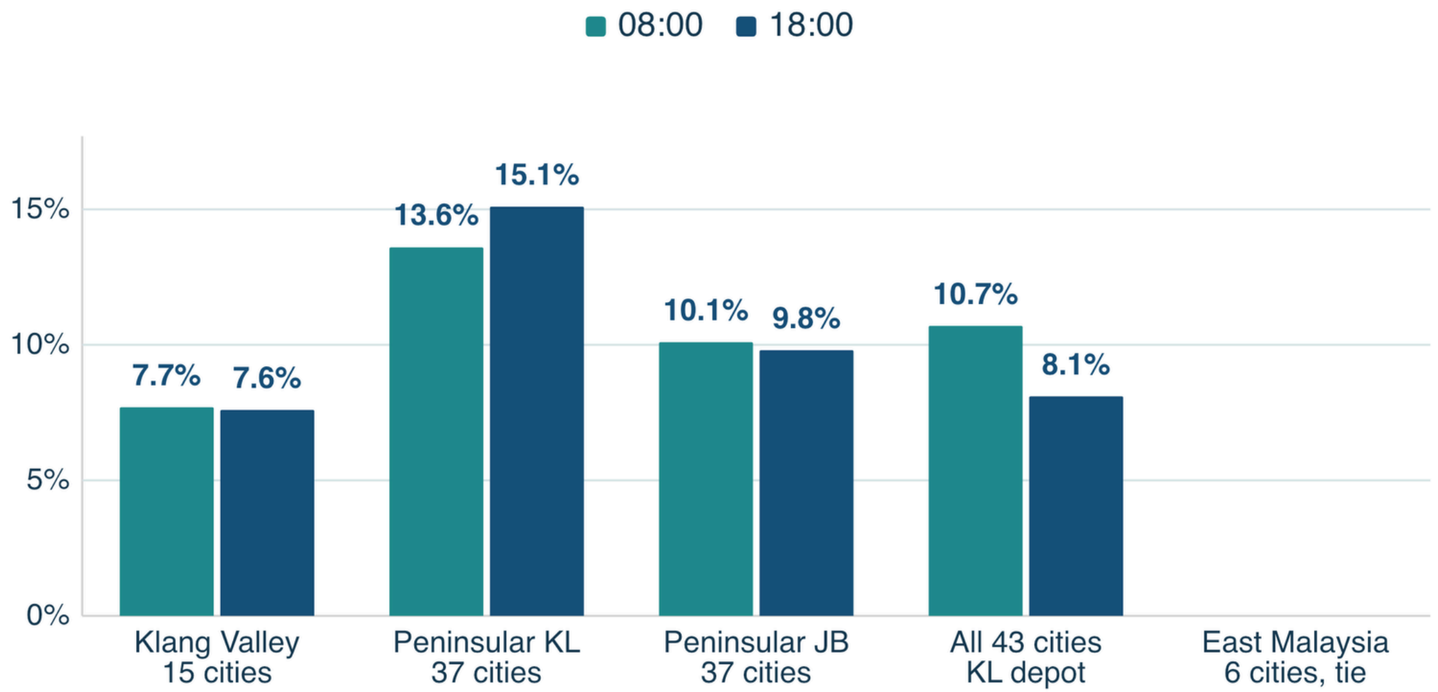
Time-only run, all 43 cities, 08:00.

Beam scored 0.0458.
2-opt scored 0.0533.

Beam drove farther but finished sooner.

How much cheaper 2-opt was

Percent lower weighted cost than beam width 100



Trade-offs and limitations

Trade-offs

Tour cost and runtime

On all 43 cities at 08:00, default 2-opt had a lower cost but took 7.88 s. Beam width 100 took 0.47 s.

One tour and many partial routes

2-opt keeps one full tour. Beam keeps up to W partial routes. Peak search memory was 29.1 KiB for 2-opt and 193.5 KiB for beam.

Best and first improvement

Best picks the cheapest move in a full scan. First takes the first one that helps. With nearest neighbour, the gaps were 0.08% and 0.16%.

Limitations

Where we know the optimum

We checked the exact optimum on small sets, up to nine cities, including six in East Malaysia. On larger maps we compare the two costs.

Cost scaling

Changing the city subset changes the scale. Changing the weights changes what the cost rewards. Compare scores within the same setup.

The 43-city map

It uses straight-line legs across the sea. Those links make sense for testing the algorithms, but a road route can’t use them.

Interesting findings

2-opt found global optima more than beam search

2-opt found the global optima 82.5% of the time, whilst beam search (width 100) only found it 24% of the time.

Beam Search was faster

On 43 cities, Beam Search (width 100) took only 0.47s whilst 2-opt took 7.88s. That's approximately 17 times faster.

Both algorithms tied when only optimizing risk

When arrival risk was the only objective, both algorithms found the global optima in all 20 nine-city trials.

X-factor: lookahead beam search

the change

Plain beam search ranks routes by cost so far, so it can't see a risky city waiting after dark.

Lookahead ranks each route by quickly finishing it with nearest neighbour, then scoring the whole trip. Only one line of the pseudocode changes.

Based on filtered beam search (Ow & Morton).

the evidence

Mean gap to optimal, plain → lookahead
(200 random 9-city instances):

W = 1: 6.07% → 1.86%
W = 20: 3.29% → 0.53%
W = 100: 2.30% → 0.23%

Trade-off: $O(W \cdot n^4)$ time, ~18× slower at W = 1.

X-factor: lookahead beam search

3.29%

before: plain beam search
mean gap to optimal, W = 20

Ranks routes by cost so far, so a risky city saved for after dark goes unnoticed until it's too late.

→

one-line change

0.53%

after: lookahead beam search
mean gap to optimal, W = 20

Finishes each route greedily before ranking it, so late-night arrivals show up early. Trade-off: ~18× slower per run.

AI usage

tools and how we used them

Tools: Cursor, ChatGPT, Claude

- Generating and debugging code
- Researching beam search and TSP variants
- Explaining concepts
- Drafting slide layouts and diagrams

our own work

- The problem model and assumptions
- Choosing beam search + 2-opt
- Final implementation decisions
- Experiment design and results

Every AI suggestion was checked against our code and brute-force results.

AI usage

Cursor and ChatGPT were used for the generation of code used in our approaches.

Prompt 1:

```
help me create an AGENTS.md file to provide context and guidelines for my teammates' AI coding agents working on this repository, so they understand our project decisions and requirements.
Create a CLAUDE.md file pointing to AGENTS.md to avoid duplicating instructions.
make sure its clear enough for my teammates and their agents to understand and continue developing the project.
```

set up AGENTS.md / CLAUDE.md so everyone's AI tools follow the same project rules

Prompt 2:

```
Look at the CS201 Data Struct project I have. Look for the sources project 2627t1 and CS201_2026_project.zip. We have processed and cleaned the data into cities_cleaned.csv. I want to create a beam search solution to the the safety-aware day/night TSP problem we discussed before. Let me know if you can see it
```

generate the Beam Search implementation

Prompt 3:

```
refer to claude.md and implement the 2-opt with test cases.
after making these changes, add a short summary of what was changed and added, what each of the changes fix or do and what is left to do for handover. state explicitly what the next person working on 2-opt can do from here
```

generate the 2-opt implementation, tests and handover notes

Q & A

thank you